Contents

Diffusion Learnings

Table of contents

Chapter 1: Image codecs — VAE → VQ-VAE → VQGAN → MoVQ (and where latent diffusion sits)

1.1 The big picture: two separate problems

1.2 The evolution of the image codec

1.3 Architecturally, these are mostly CNN/ResNet autoencoders

1.4 What makes a VAE a VAE?

1.5 The VAE problem: KL can become too successful (posterior collapse)

1.6 VQ-VAE's radically different idea: make z discrete

1.7 The VQ-VAE training problem: nearest neighbour kills the gradient

1.8 Straight-through estimator: lie in backward, not forward

1.9 VQ-VAE has three learning jobs

1.10 Why codebook + commitment separately?

1.11 EMA gives an even cleaner interpretation of codebook learning

1.12 Why pixel MSE alone can produce blurry reconstruction

1.13 Three complementary reconstruction pressures

1.14 LPIPS from beginning to end

1.15 PatchGAN is a CNN critic of local realism

1.16 VQGAN = VQ representation + perceptual realism

1.17 MoVQ: what was left to improve?

1.18 MoVQ also injects the codes repeatedly into the decoder

1.19 The important latent distinction (three different "latents")

1.20 This explains the evolution of the actual generation systems

1.21 Why the DALL·E 2 prior can help diversity

1.22 The final mental map

Chapter 2: Training mechanics — normalization, CNNs, and the Adam/AdamW family

2.1 CNN translation behavior: equivariance vs invariance

2.2 Normalization: BatchNorm / GroupNorm / LayerNorm / RMSNorm

2.3 Adam: an adaptive per-coordinate learning rate

2.4 Why large normalized weights can still be undesirable

2.5 L2 regularization

2.6 Why AdamW exists

2.7 Weight decay does not normally flip signs

Chapter 3: Diffusion sampling — DDPM, Spaced DDPM, DDIM

3.1 Forward vs generation, and the DDPM chain

3.2 Collapsing Gaussian steps → Spaced DDPM

3.3 DDIM: a deterministic trajectory between arbitrary noise levels

3.4 Spaced DDPM vs DDIM, and the modern picture

Chapter 4: The Diffusion Handbook (Parts I–X)

Chapter 5: Image conditioning — vision encoder + image projector

5.1 Two kinds of conditioning: semantic vs pixel-aligned

5.2 Image conditioning as tokens: $I = P_{\text{img}}(E_{\text{vision}}(x_0))$

5.3 Why encode the *clean* image, and why a projector at all

5.4 Where I actually enters: decoupled cross-attention

5.5 Tokens vs channel-concat: choosing by whether the pixels line up

Chapter 6: Flow matching and few-step generators — FM → Consistency → FMM → AYF → MeanFlow → iMF

6.0 One notation for the whole family

6.1 Flow matching: a probability path from data to noise

6.2 Conditional velocity vs marginal velocity

6.3 Why conditional flow matching works (the MSE argument)

6.4 The continuity equation: why the ODE transports p_t

6.5 Straight interpolant trajectory vs probability-flow trajectory

6.6 Sampling, and the motivation for few-step methods

6.7 Consistency models: predict the endpoint directly

6.8 Consistency distillation and consistency training

6.9 Flow map matching: the two-time solution operator

6.10 The Lagrangian identity and FMM-LMD

6.11 The Eulerian identity and FMM-EMD

6.12 The JVP tangent $(v_s, 1, 0)$, and why it is expensive

6.13 Direct FMM from scratch: cycle + dynamics

6.14 AYF: stop-gradient the residual, use it as a direction

6.15 MeanFlow: learn the average velocity

6.16 The MeanFlow identity and its JVP target

6.17 iMF: a network-independent, guidance-flexible target

6.18 The unifying view: what object does the network learn?

Chapter 7: Guidance — tilting a generative prior toward a reward

7.1 The distribution-level goal: reward minus KL

7.2 The optimal tilted distribution $q^* \propto p_{\text{gen}} e^{R/\beta}$

7.3 Classifier guidance and CFG

7.4 Universal guidance: reward on the predicted clean image

7.5 Manifold-preserving guidance (MPGD)

7.6 Optimizing the starting noise

7.7 Fine-tuning the generator (AlignProp)

7.8 Flow-map terminal guidance: a short backprop graph

Chapter 8: Application — maximizing fMRI ROI activation with generative priors

8.1 The ROI reward from a differentiable brain encoder

8.2 Method A — BrainDiVE-style step-by-step diffusion guidance

8.3 Method B — specificity: "X but not Y"

8.4 Normalizing voxel responses

8.5 Methods C–F — manifold guidance, noise optimization, flow-map, LoRA

8.6 Method G — optimize a distribution, not a single image

8.7 The real adversary: encoder reward hacking

8.8 A three-level plan, and the master gradient

Chapter 9: MeanFlow and iMF in depth — moving targets, JVP noise, and guidance intervals

9.1 The moving-target problem: stop-gradient ≠ fixed target

9.2 Conditional and unconditional FM share the label $\epsilon - x$

9.3 Second derivatives are not inherently noisy

9.4 Noisy v_c *inside* the JVP: $J\eta$ and its covariance

9.5 The Jacobian SVD: which directions amplify noise

9.6 SVD vs PCA

9.7 Label noise vs tangent noise: why iMF's reversal helps

9.8 CFG inside original MeanFlow

9.9 Two independent axes: conditional/unconditional vs $r = t$ / $r < t$

9.10 Why original MF bakes in the CFG scale

9.11 iMF makes ω an input: embedding and in-context conditioning

9.12 The guidance interval $[t_{\min}, t_{\max}]$ and 1-NFE

9.13 Guidance interval vs number of inference steps

9.14 The full FM → MF → modified MF → iMF ladder

Diffusion Learnings

A running, study-from-cold notebook. One session → one chapter. Every equation in LaTeX, every idea built up from first principles with worked examples.

Table of contents

Chapter 1 — Image codecs: VAE → VQ-VAE → VQGAN → MoVQ (and where latent diffusion sits)

- 1.1 The big picture: two separate problems
- 1.2 The evolution of the image codec
- 1.3 Architecturally, these are mostly CNN/ResNet autoencoders
- 1.4 What makes a VAE a VAE?
- 1.5 The VAE problem: KL can become too successful (posterior collapse)
- 1.6 VQ-VAE's radically different idea: make z discrete
- 1.7 The VQ-VAE training problem: nearest neighbour kills the gradient
- 1.8 Straight-through estimator: lie in backward, not forward
- 1.9 VQ-VAE has three learning jobs
- 1.10 Why codebook + commitment separately?
- 1.11 EMA gives an even cleaner interpretation of codebook learning
- 1.12 Why pixel MSE alone can produce blurry reconstruction
- 1.13 Three complementary reconstruction pressures
- 1.14 LPIPS from beginning to end
- 1.15 PatchGAN is a CNN critic of local realism
- 1.16 VQGAN = VQ representation + perceptual realism
- 1.17 MoVQ: what was left to improve?
- 1.18 MoVQ also injects the codes repeatedly into the decoder
- 1.19 The important latent distinction (three different "latents")
- 1.20 This explains the evolution of the actual generation systems
- 1.21 Why the DALL·E 2 prior can help diversity
- 1.22 The final mental map

Chapter 2 — Training mechanics: normalization, CNNs, and the Adam/AdamW family

- 2.1 CNN translation behavior: equivariance vs invariance
- 2.2 Normalization: BatchNorm / GroupNorm / LayerNorm / RMSNorm
- 2.3 Adam: an adaptive per-coordinate learning rate
- 2.4 Why large normalized weights can still be undesirable
- 2.5 L2 regularization

- 2.6 Why AdamW exists
- 2.7 Weight decay does not normally flip signs

Chapter 3 — Diffusion sampling: DDPM, Spaced DDPM, DDIM

- 3.1 Forward vs generation, and the DDPM chain
- 3.2 Collapsing Gaussian steps → Spaced DDPM
- 3.3 DDIM: a deterministic trajectory between arbitrary noise levels
- 3.4 Spaced DDPM vs DDIM, and the modern picture

Chapter 4 — The Diffusion Handbook (Parts I–X) *(pending source recovery — see note)*

Chapter 5 — Image conditioning: vision encoder + image projector

- 5.1 Two kinds of conditioning: semantic vs pixel-aligned
- 5.2 Image conditioning as tokens: $I = P_{\text{img}}(E_{\text{vision}}(x_0))$
- 5.3 Why encode the *clean* image, and why a projector at all
- 5.4 Where I actually enters: decoupled cross-attention
- 5.5 Tokens vs channel-concat: choosing by whether the pixels line up

Chapter 6 — Flow matching and few-step generators: FM → Consistency → FMM → AYF → MeanFlow → iMF

- 6.0 One notation for the whole family
- 6.1 Flow matching: a probability path from data to noise
- 6.2 Conditional velocity vs marginal velocity
- 6.3 Why conditional flow matching works (the MSE argument)
- 6.4 The continuity equation: why the ODE transports p_t
- 6.5 Straight interpolant trajectory vs probability-flow trajectory
- 6.6 Sampling, and the motivation for few-step methods
- 6.7 Consistency models: predict the endpoint directly
- 6.8 Consistency distillation and consistency training
- 6.9 Flow map matching: the two-time solution operator
- 6.10 The Lagrangian identity and FMM-LMD
- 6.11 The Eulerian identity and FMM-EMD
- 6.12 The JVP tangent $(v_s, 1, 0)$, and why it is expensive
- 6.13 Direct FMM from scratch: cycle + dynamics
- 6.14 AYF: stop-gradient the residual, use it as a direction
- 6.15 MeanFlow: learn the average velocity
- 6.16 The MeanFlow identity and its JVP target
- 6.17 iMF: a network-independent, guidance-flexible target
- 6.18 The unifying view: what object does the network learn?

Chapter 7 — Guidance: tilting a generative prior toward a reward

- 7.1 The distribution-level goal: reward minus KL
- 7.2 The optimal tilted distribution $q^* \propto p_{\text{gen}} e^{R/\beta}$
- 7.3 Classifier guidance and CFG
- 7.4 Universal guidance: reward on the predicted clean image
- 7.5 Manifold-preserving guidance (MPGD)
- 7.6 Optimizing the starting noise
- 7.7 Fine-tuning the generator (AlignProp)
- 7.8 Flow-map terminal guidance: a short backprop graph

Chapter 8 — Application: maximizing fMRI ROI activation with generative priors

- 8.1 The ROI reward from a differentiable brain encoder
- 8.2 Method A — BrainDiVE-style step-by-step diffusion guidance
- 8.3 Method B — specificity: "X but not Y"
- 8.4 Normalizing voxel responses
- 8.5 Methods C–F — manifold guidance, noise optimization, flow-map, LoRA
- 8.6 Method G — optimize a distribution, not a single image
- 8.7 The real adversary: encoder reward hacking
- 8.8 A three-level plan, and the master gradient

Chapter 9 — MeanFlow and iMF in depth: moving targets, JVP noise, and guidance intervals

- 9.1 The moving-target problem: stop-gradient ≠ fixed target
- 9.2 Conditional and unconditional FM share the label $\epsilon - x$
- 9.3 Second derivatives are not inherently noisy
- 9.4 Noisy v_c *inside* the JVP: $J\eta$ and its covariance
- 9.5 The Jacobian SVD: which directions amplify noise
- 9.6 SVD vs PCA
- 9.7 Label noise vs tangent noise: why iMF's reversal helps
- 9.8 CFG inside original MeanFlow
- 9.9 Two independent axes: conditional/unconditional vs $r = t$ / $r < t$
- 9.10 Why original MF bakes in the CFG scale
- 9.11 iMF makes ω an input: embedding and in-context conditioning
- 9.12 The guidance interval $[t_{\min}, t_{\max}]$ and 1-NFE
- 9.13 Guidance interval vs number of inference steps
- 9.14 The full FM → MF → modified MF → iMF ladder

Chapter 1: Image codecs — VAE → VQ-VAE → VQGAN → MoVQ (and where latent diffusion sits)

This chapter builds the clean mental model tying together the whole image-codec story — from VAE to VQ-VAE to VQGAN to MoVQ — plus the losses, straight-through gradients, LPIPS/GAN, and *why* these systems evolved the way they did. The recurring theme: modern image generators separate **compressing images** from **generating good compressed representations**, and almost every design choice below is an answer to "how should we compress an image so reconstruction is excellent, the latent is easy to model, and generation is cheap?"

1.1 The big picture: two separate problems

Modern image generation often separates:

$$\boxed{\text{compress/decompress images}}$$

from:

$$\boxed{\text{generate good compressed representations}}.$$

The first is the image codec:

$$x \xrightarrow{E} z \xrightarrow{D} \hat{x}.$$

The second can be diffusion, flow matching, autoregression, etc.:

$$\epsilon \xrightarrow[\text{text}]{\text{generative model}} z \xrightarrow{D} x.$$

This distinction explains Stable Diffusion:

$$\text{text} + \text{noise} \xrightarrow{\text{latent diffusion}} z \xrightarrow{\text{VAE decoder}} x.$$

And Kandinsky 2.1:

$$\text{text} \to z_I^{CLIP} \to \text{diffusion in MoVQ latent space} \to z^{MoVQ} \to \text{MoVQ}$$

MoVQ isn't doing the text-to-image reasoning. It's primarily an image compressor/decompressor.

1.2 The evolution of the image codec

The easiest history to remember is:

$$\boxed{\text{VAE} \to \text{VQ-VAE} \to \text{VQGAN} \to \text{MoVQ}}$$

but these aren't simply newer versions replacing one another. They make different bottleneck choices.

VAE

$$x \xrightarrow{\text{CNN encoder}} (\mu, \sigma)$$

sample:

$$z = \mu + \sigma\epsilon, \qquad \epsilon \sim N(0, I)$$

then:

$$z \xrightarrow{\text{CNN decoder}} \hat{x}.$$

Continuous stochastic latent.

VQ-VAE

$$x \xrightarrow{\text{CNN encoder}} z_e \xrightarrow{\text{nearest codebook vector}} z_q \xrightarrow{\text{CNN decoder}} \hat{x}.$$

Discrete latent.

VQGAN

Basically:

$$\boxed{\text{stronger VQ-VAE} + \text{perceptual reconstruction} + \text{adversarial trainin}}$$

to get much better perceptual image quality.

MoVQ

Builds on the VQGAN family but improves how much information discrete codes can represent and how the decoder uses them.

1.3 Architecturally, these are mostly CNN/ResNet autoencoders

For images, the classic structure is approximately:

$$\begin{gathered} 256 \times 256 \times 3 \\ \downarrow \quad \text{Conv} + \text{ResBlocks} \\ 128 \times 128 \times C \\ \downarrow \end{gathered}$$

$$64 \times 64 \times 2C$$

$$\downarrow$$

$$32 \times 32 \times 4C$$

$$\downarrow$$

$$\boxed{z}.$$

Then the decoder mirrors this:

$$z \to 32^2 \to 64^2 \to 128^2 \to 256^2 \times 3.$$

A typical residual block looks approximately like:

$$F \to GN \to SiLU/ReLU \to Conv_{3\times 3} \to GN \to activation \to Conv$$

plus:

$$+ F$$

for the residual connection.

Different models use somewhat different exact blocks, but that's the overall architecture. Transformers can and increasingly do appear in image tokenizers/codecs, but CNNs remain attractive because the codec mostly needs efficient local processing. Global self-attention over every high-resolution spatial position is expensive.

Hence the very natural division:

$$\boxed{\text{efficient CNN/hybrid codec} + \text{powerful Transformer/DiT in compres}}$$

1.4 What makes a VAE a VAE?

The encoder doesn't produce one deterministic z. It produces a distribution:

$$q_\phi(z|x) = N(\mu(x), \mathrm{diag}(\sigma^2(x))).$$

Then:

$$z = \mu + \sigma\epsilon.$$

There are two competing objectives. Reconstruct:

$$L_{\text{rec}}(x, \hat{x})$$

while making latent distributions resemble:

$$p(z) = N(0, I).$$

So:

$$\boxed{L_{\text{VAE}} = L_{\text{rec}} + \beta\, D_{KL}(q_\phi(z|x)\|N(0,I)).}$$

The KL term makes latent space organized and sampleable. Instead of every image being mapped somewhere arbitrary:

```
cat      -> [500, -82, 1300, ...]
dog      -> [-9000, 72, 4, ...]
chair    -> [18273, -3, ...]
```

we encourage the aggregate representation toward a known, manageable region. This is particularly useful when another generative model must operate in that latent space.

1.5 The VAE problem: KL can become too successful (posterior collapse)

Suppose the decoder is powerful enough to model x without using z. Then compare:

$$\text{use } z: \quad L_{\text{rec}} = 1.0,\ KL = 4$$

versus:

$$\text{ignore } z: \quad L_{\text{rec}} = 1.2,\ KL = 0.$$

The second has lower total loss. Therefore:

$$q(z|x) \to p(z).$$

But then $q(z|x)$ barely depends on x, so z contains little information. That's:

$$\boxed{\text{posterior collapse}}.$$

This is especially problematic with very powerful autoregressive decoders.

Compact recap (the tug-of-war). Posterior collapse in VAEs: KL pushes $q(z|x)$ toward the prior, which can erase information about x. Reconstruction losses push the opposite way by requiring z to preserve enough information to rebuild the input. LPIPS helps preserve perceptually meaningful structure; GAN loss pushes reconstructions toward looking like real images.

1.6 VQ-VAE's radically different idea: make z discrete

Create a dictionary:

$$\boxed{\mathcal{E} = \{e_1, e_2, \ldots, e_K\}}$$

where $e_k \in \mathbb{R}^D$. The encoder outputs:

$$z_e \in \mathbb{R}^{h \times w \times D}.$$

At every spatial position $z_e[i,j] \in \mathbb{R}^D$, find:

$$\boxed{k^* = \arg\min_k \|z_e[i,j] - e_k\|^2.}$$

Then replace $z_e[i,j]$ with:

$$\boxed{z_q[i,j] = e_{k^*}.}$$

So the decoder never sees the arbitrary continuous encoder vector. It sees a member of a finite learned vocabulary. You can think of the codes as learned visual words, although they need not correspond cleanly to human concepts.

1.7 The VQ-VAE training problem: nearest neighbour kills the gradient

The quantizer is:

$$Q(z_e) = e_{\arg\min_k \|z_e - e_k\|}.$$

It's piecewise constant. If $e_1 = 0,\ e_2 = 10$, then approximately:

$$Q(z) = \begin{cases} 0 & z < 5 \\ 10 & z > 5. \end{cases}$$

Almost everywhere:

$$\boxed{\frac{\partial Q}{\partial z} = 0.}$$

Therefore the true reconstruction gradient would give:

$$\frac{\partial L}{\partial z_e} = \frac{\partial L}{\partial z_q} \underbrace{\frac{\partial z_q}{\partial z_e}}_{0} = 0.$$

The encoder couldn't learn from reconstruction.

1.8 Straight-through estimator: lie in backward, not forward

Forward remains genuinely discrete:

$$\boxed{z_q = Q(z_e)}.$$

But backward pretends:

$$\boxed{\frac{\partial z_q}{\partial z_e} \approx I.}$$

Therefore:

$$\boxed{\frac{\partial L}{\partial z_e} \approx \frac{\partial L}{\partial z_q}.}$$

This is not the mathematically correct local derivative — it's a useful surrogate. The decoder effectively says: *if my latent input moved this direction, reconstruction would improve.* STE copies that signal to the encoder. Then z_e moves continuously until potentially crossing a nearest-neighbour boundary:

$$e_{17} \to e_{42}.$$

So the surrogate gradient eventually changes the actual discrete decision. That's why it works despite being biased.

1.9 VQ-VAE has three learning jobs

This is the cleanest way to remember its losses. We have three components:

$$\boxed{\text{encoder}} \qquad \boxed{\text{codebook}} \qquad \boxed{\text{decoder}}.$$

Job 1: reconstruct the image

$$L_{\text{rec}} = \|x - D(z_q)\|^2.$$

This trains the decoder directly and the encoder through STE.

Job 2: make dictionary vectors follow encoder clusters

$$\boxed{L_{\text{code}} = \|\operatorname{sg}[z_e] - e_k\|^2.}$$

Only e_k gets this gradient.

Job 3: make encoder commit to the dictionary

$$\boxed{L_{\text{commit}} = \beta\|z_e - \operatorname{sg}[e_k]\|^2.}$$

Only the encoder gets this gradient. Together:

$$\boxed{L_{\text{VQ}} = L_{\text{rec}} + L_{\text{code}} + \beta L_{\text{commit}}.}$$

1.10 Why codebook + commitment separately?

Because they represent opposite responsibilities:

$$\boxed{e_k \to z_e}$$

means *dictionary: move toward the encoder representations assigned to you.* While:

$$\boxed{z_e \to e_k}$$

means *encoder: don't move arbitrarily far from the discrete representation your decoder actually receives.* The second can be weaker ($\beta < 1$). This means: give the encoder freedom to discover useful representations, let the dictionary chase those representations strongly, but keep enough commitment that encoder outputs and actual quantized inputs don't diverge.

If β is huge: the encoder gets trapped around existing codes. If β is too small: the encoder can outrun/drift away from its codebook. That's why separating the two is useful.

1.11 EMA gives an even cleaner interpretation of codebook learning

Instead of gradient-descent updating code e_k, collect encoder outputs assigned to it:

$$z_e^{(1)}, z_e^{(2)}, \dots$$

Calculate their mean $\bar{z}_k$. Then approximately:

$$\boxed{e_k \leftarrow \gamma e_k + (1-\gamma)\bar{z}_k.}$$

So the codebook behaves like online k-means. Then responsibilities become very clean:

$$\boxed{\text{encoder} \xleftarrow{\text{gradient}} L_{\text{rec}} + L_{\text{commit}}}$$

$$\boxed{\text{decoder} \xleftarrow{\text{gradient}} L_{\text{rec}}}$$

$$\boxed{\text{codebook} \xleftarrow{\text{EMA}} \text{encoder cluster centers.}}$$

1.12 Why pixel MSE alone can produce blurry reconstruction

Compression means $x \to z$ throws away information. Suppose after compression the decoder can't know whether the original contained texture A, B, or C. For MSE:

$$\hat{x} = \arg\min_y E[\|X - y\|^2 \mid z].$$

The optimum is:

$$\boxed{\hat{x} = E[X \mid z].}$$

That's the conditional mean. If hair could occur at several slightly different locations, averaging them can produce:

$$\boxed{\text{smooth/blurry hair}}.$$

This isn't because MSE is a bad metric mathematically. It's because under uncertainty, squared error rewards averaging. With unlimited information capacity and perfect reconstruction, this issue could largely disappear. Compression makes it important.

Compact recap. Why MSE blurs: if the latent loses information and one z is compatible with multiple possible images, minimizing squared error gives the conditional mean $E[x|z]$. Averaging multiple plausible sharp images can look blurry. LPIPS changes the geometry by comparing deep visual features rather than exact pixels, so a coherent sharp structure can sometimes be preferable to a pixelwise average. GAN strengthens the pressure toward realistic-looking outputs.

1.13 Three complementary reconstruction pressures

This became a major idea in VQGAN/perceptual autoencoders.

Pixel loss

$$L_{\text{pixel}} = \|x - \hat{x}\|_1 \quad \text{or} \quad \|x - \hat{x}\|_2^2.$$

Asks: "*Are these exact pixel values close?*" — an excellent anchor to the particular target.

LPIPS

$$L_{\text{LPIPS}} = d_{\text{perceptual}}(x, \hat{x}).$$

Asks: "*Does this perceptually resemble this target?*" — allows some pixel disagreement while preserving perceptually important structure.

GAN

$$L_{\text{GAN}}$$

asks: "*Does this look like it came from the real-image distribution?*" — it does not ask directly whether every hair matches the original hair. So:

$$\boxed{\text{pixel} = \text{exact target anchor}}$$

$$\boxed{\text{LPIPS} = \text{perceptual target similarity}}$$

$$\boxed{\text{GAN} = \text{realism/manifold pressure.}}$$

This is why LPIPS and GAN are not redundant.

1.14 LPIPS from beginning to end

Take $x, \hat{x}$. Run both through the same frozen pretrained visual network:

$$x \to \phi_1(x), \ldots, \phi_L(x)$$

$$\hat{x} \to \phi_1(\hat{x}), \ldots, \phi_L(\hat{x}).$$

At layer l: $\phi_l(x) \in \mathbb{R}^{H_l \times W_l \times C_l}$. Normalize feature vectors along channels:

$$\tilde{\phi}_l(x)_{hw} = \frac{\phi_l(x)_{hw}}{\|\phi_l(x)_{hw}\|}.$$

Calculate:

$$\Delta_{lhwc} = \tilde{\phi}_l(x)_{hwc} - \tilde{\phi}_l(\hat{x})_{hwc}.$$

Square and weight channels:

$$d_{lhw} = \sum_c w_{lc}^2 \Delta_{lhwc}^2.$$

Average spatially:

$$D_l = \frac{1}{H_l W_l} \sum_{h,w} d_{lhw}.$$

Sum layers:

$$\boxed{LPIPS(x, \hat{x}) = \sum_l D_l.}$$

The w_l's were calibrated using human perceptual judgments so that the resulting distance better predicts which image humans perceive as closer to a reference. During VQGAN/VAE training, LPIPS is frozen. But:

$$\boxed{\text{frozen} \neq \text{no gradient.}}$$

Gradient flows:

$$L_{\text{LPIPS}} \to \hat{x} \to \text{decoder} \to \cdots$$

without updating LPIPS's own weights.

Compact recap. LPIPS: pass both images through a frozen CNN, take feature vectors at several layers and spatial locations, normalize those vectors, compare their differences, apply learned per-channel perceptual weights, average over space, and sum across layers. Because normalized squared Euclidean distance is closely related to cosine distance, LPIPS is effectively comparing feature direction with learned importance across channels.

CNN translation behavior. Convolutional feature maps are mainly translation-equivariant: shift the image, and the feature map shifts. Invariance appears when location is deliberately discarded, such as through global pooling. LPIPS keeps spatial feature maps, so it is not fully translation invariant.

1.15 PatchGAN is a CNN critic of local realism

Input: $256 \times 256 \times 3$. The CNN discriminator produces progressively richer features:

$$256^2 \times 3 \to 128^2 \times 64 \to 64^2 \times 128 \to 32^2 \times 256 \to \cdots$$

At one spatial location there may be $F[i, j] \in \mathbb{R}^{512}$. That's not just one number — it is a rich description of a large receptive field in the original image. A final convolution turns $\mathbb{R}^{512} \to \mathbb{R}$ at every location. So the final discriminator output is something like:

$$D(x) \in \mathbb{R}^{30 \times 30}.$$

Every element asks approximately:

> "Does the corresponding receptive field look locally real?"

The patches overlap — you don't manually cut the image — and convolutional weight sharing means the same learned realism detectors operate across the image.

1.16 VQGAN = VQ representation + perceptual realism

This is the conceptual jump.

VQ-VAE

$$x \to E \to z_e \to VQ \to z_q \to D \to \hat{x}$$

primarily reconstruction + VQ losses.

VQGAN — same broad architecture:

$$x \to E \to z_e \to VQ \to z_q \to D \to \hat{x}$$

but train reconstruction using something closer to:

$$\boxed{L = L_{\text{pixel/perceptual}} + L_{\text{codebook}} + L_{\text{commitment}} + \lambda L_{\text{GAN}}.}$$

Now the codec is allowed to use its limited bits for perceptually important information rather than obsessing entirely over exact pixel errors.

1.17 MoVQ: what was left to improve?

A VQGAN might map nearby/similar encoder vectors to the same code:

$$z_e^A \to e_{17} \qquad z_e^B \to e_{17}.$$

Then the decoder sees exactly e_{17} in both places. This can limit local representational richness and contribute to repeated patterns/artifacts. MoVQ addresses this in two notable ways.

More combinations per spatial position. Instead of one 256-D vector selecting one code, split:

$$256 = 64 + 64 + 64 + 64.$$

Then:

$$z_e[i, j] = [z^1, z^2, z^3, z^4].$$

Each selects independently (k_1, k_2, k_3, k_4). If $K = 1024$, you have:

$$\boxed{K^4}$$

possible combinations at each spatial location without storing a literal K^4-entry codebook.

1.18 MoVQ also injects the codes repeatedly into the decoder

Instead of only $z_q \to D$, it uses:

$$\boxed{F_l = \gamma_l(z_q) \odot GN(F_{l-1}) + \beta_l(z_q).}$$

Here F_{l-1} is the decoder's current feature tensor. GroupNorm normalizes it. Then learned 1×1 convolutions produce spatially varying $\gamma_l(z_q)$ and $\beta_l(z_q)$: γ scales features, β shifts features. Therefore the discrete representation repeatedly tells the decoder: "*at this spatial region, amplify/suppress/shift these channels this way.*" It's much stronger conditioning than simply giving z_q once at the bottom.

1.19 The important latent distinction (three different "latents")

This was one of the biggest conceptual points from the entire discussion. There are at least three completely different things people casually call "latent."

CLIP image latent z_I^{CLIP} — global semantic representation:

$$\boxed{\text{"What kind of image/content is this?"}}$$

Not designed to reconstruct every pixel.

VAE / VQ / MoVQ latent $z^{VAE},\ z^{VQ},\ z^{MoVQ}$ — spatial compressed image representation:

$$\boxed{\text{"Compressed information from which I can reconstruct the image."}}$$

Diffusion latent state z_t — a noisy intermediate state:

$$z_t = \sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon.$$

Totally different from the other two.

1.20 This explains the evolution of the actual generation systems

DALL·E 1 — no diffusion:

$$\text{text} \rightarrow \text{autoregressive Transformer} \rightarrow \text{discrete dVAE image tokens} -$$

GLIDE — no image autoencoder bottleneck for the main generation:

$$\text{text} \rightarrow \text{pixel diffusion} \rightarrow 64^2 \rightarrow \text{diffusion SR}.$$

DALL·E 2 / unCLIP:

$$\text{text} \xrightarrow{CLIP} z_T \xrightarrow{\text{diffusion prior}} z_I^{CLIP}$$

then:

$$z_I^{CLIP} \rightarrow \text{pixel diffusion at } 64^2 \rightarrow \text{diffusion SR} \rightarrow 1024^2.$$

So DALL·E 2's main image diffusion happens in pixels, not a VAE spatial latent like Stable Diffusion.

Karlo — similar unCLIP concept:

$$\text{text} \rightarrow CLIP\ image\ prior \rightarrow 64^2\ pixel\ diffusion \rightarrow fast\ SR.$$

Stable Diffusion — different:

$$x \to \boxed{\text{VAE latent}}$$

and diffusion learns $\text{t} + \text{noise} \to z^{VAE}$, then:

$$z^{VAE} \xrightarrow{\text{VAE decoder}} x.$$

The expensive iterative process happens in compressed space.

Kandinsky 2.1 — combines ideas:

$$\text{text} \to z_I^{CLIP}$$

then:

$$z_I^{CLIP} + \text{text} \to \text{diffusion in MoVQ spatial latent}$$

then:

$$z^{MoVQ} \xrightarrow{\text{MoVQ decoder}} x.$$

So conceptually:

$$\boxed{\text{unCLIP semantic prior} + \text{Stable-Diffusion-style latent generation} +}$$

1.21 Why the DALL·E 2 prior can help diversity

A direct model learns $p(x|c)$. DALL·E 2 explicitly factorizes:

$$\boxed{p(x|c) = \int p(x|z_I, c)\, p(z_I|c)\, dz_I.}$$

So for "a corgi on a beach" the prior can first sample:

$$z_I^1 = \text{close-up} \qquad z_I^2 = \text{running through water} \qquad z_I^3 = \text{beside a surfb}$$

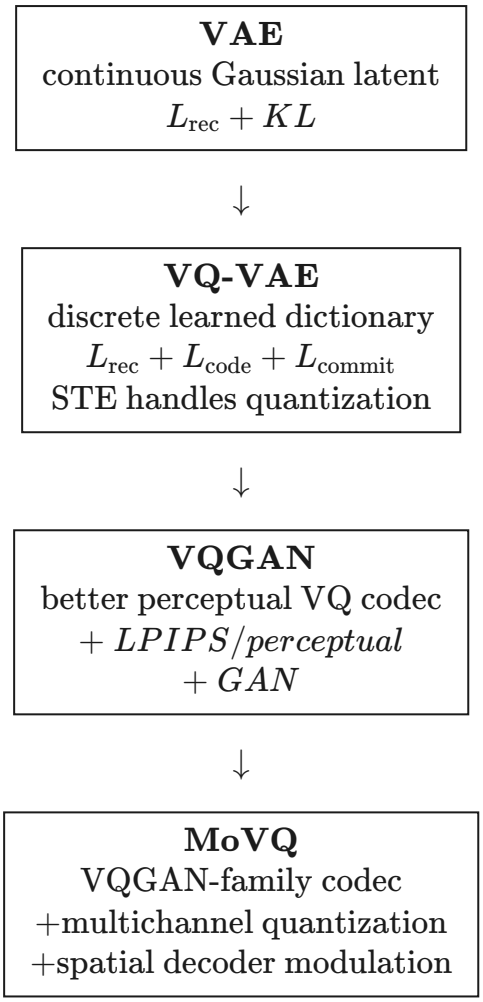
Then the decoder renders the selected interpretation. Conceptually:

$$\boxed{\text{prior chooses visual interpretation}}$$

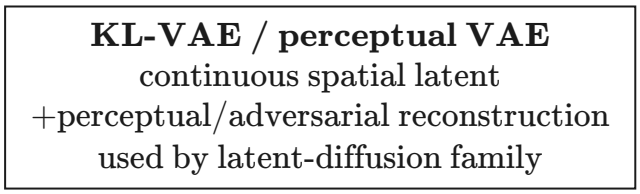
then the decoder realizes it. A sufficiently powerful direct model can learn this implicitly, so the prior is not mathematically necessary. It is an architectural factorization / inductive bias.

1.22 The final mental map

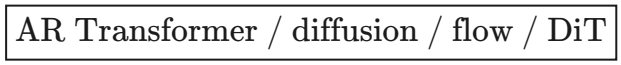
If you remember only this, remember:



while the alternative branch is:

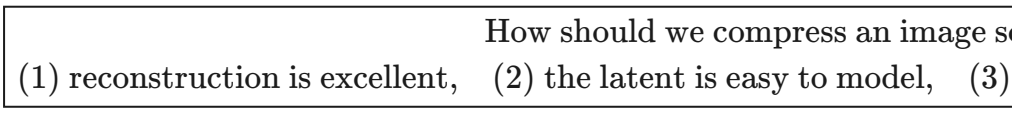


And on top of whichever codec you choose, you can train a powerful generative model:



to learn the distribution of those latents.

The conceptual evolution is therefore not simply "VAE got replaced by VQ-VAE got replaced by VQGAN got replaced by MoVQ." It is:



VAE and VQ-style codecs give different answers to that tradeoff, and modern image generators build the expensive semantic/generative model on top of that compressed representation rather than repeatedly reasoning over full-resolution RGB pixels.

> **Key takeaway.** Separate the two problems: a codec ($x \to z \to \hat{x}$) and a generator over z. VAE regularizes a continuous latent with KL (risking posterior collapse); VQ-VAE swaps to a discrete codebook, needing the straight-through estimator plus codebook + commitment losses; VQGAN adds LPIPS + PatchGAN so limited bits buy *perceptual* quality; MoVQ enriches codes (multi-channel quantization $\to K^4$ combinations) and modulates the decoder with them ($F_l = \gamma_l(z_q) \odot GN(F_{l-1}) + \beta_l(z_q)$). Never conflate the three "latents" — CLIP semantic z_I^{CLIP}, spatial codec $z^{VAE/VQ}$, and noisy diffusion state z_t.

Chapter 2: Training mechanics — normalization, CNNs, and the Adam/AdamW family

This chapter collects the optimizer and normalization machinery that sits under almost every model in this notebook. The through-line: normalization decides *what statistic* a layer is invariant to, and the optimizer decides *how* a parameter moves — and the two interact (normalized nets care about weight *direction*, which is why weight decay is handled the way it is).

2.1 CNN translation behavior: equivariance vs invariance

Convolutional feature maps are mainly translation-equivariant: shift the image, and the feature map shifts. Invariance appears when location is deliberately discarded, such as through global pooling. LPIPS keeps spatial feature maps, so it is not fully translation invariant.

2.2 Normalization: BatchNorm / GroupNorm / LayerNorm / RMSNorm

BatchNorm normalizes each channel using statistics across batch and spatial positions. GroupNorm avoids dependence on the batch by normalizing groups of channels within each image across channel-group, height, and width. LayerNorm normalizes each token across its hidden dimensions. RMSNorm mainly normalizes vector magnitude and skips explicit mean subtraction.

2.3 Adam: an adaptive per-coordinate learning rate

For each parameter, Adam tracks

$$m_t \approx \text{smoothed gradient}$$

and

$$v_t \approx \text{smoothed squared gradient}.$$

The update is roughly

$$-\eta \frac{m_t}{\sqrt{v_t}}.$$

So Adam behaves like an adaptive learning rate per parameter coordinate. It normalizes gradient scale; it does not directly control the size of the parameter itself.

2.4 Why large normalized weights can still be undesirable

If normalization makes w and $100w$ behave almost identically in the forward pass, the norm of w can drift. But for a fixed-size update Δw, a larger $\|w\|$ means a smaller change in its direction:

$$\frac{\|\Delta w\|}{\|w\|} \downarrow .$$

In normalized networks, direction can matter more than magnitude, so this can effectively slow learning.

2.5 L2 regularization

Adding

$$\frac{\lambda}{2}\|w\|^2$$

to the loss gives gradient

$$\lambda w,$$

because

$$\nabla_w \frac{\lambda}{2}\|w\|^2 = \lambda w.$$

With plain SGD this creates

$$w_{t+1} = w_t - \eta g - \eta\lambda w_t.$$

2.6 Why AdamW exists

If you put λw inside Adam's gradient, Adam also rescales the regularization according to past gradient statistics m, v. Then equal-sized weights can shrink by different amounts. AdamW separates them:

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta\lambda w_t.$$

So gradient adaptation depends on gradient history, while weight shrinkage depends directly on w.

2.7 Weight decay does not normally flip signs

Ignoring the task gradient,

$$w_{t+1} = (1 - \eta\lambda) w_t.$$

For the usual case $0 < \eta\lambda \ll 1$, this gently shrinks w toward zero without crossing it.

> **Key takeaway.** Adam adapts the *step per coordinate* from gradient history ($m_t/\sqrt{v_t}$); weight decay controls *weight magnitude* directly. AdamW keeps those two jobs separate ($-\eta\lambda w_t$ applied outside the adaptive term) so that in a normalized network — where direction matters more than magnitude — shrinkage is uniform and predictable rather than rescaled by gradient statistics.

Chapter 3: Diffusion sampling — DDPM, Spaced DDPM, DDIM

This chapter distinguishes two families of "skip-ahead" samplers that look identical on a trajectory plot but arise from completely different reasoning: a *shorter stochastic Markov chain* (Spaced DDPM) versus a *deterministic change-of-variables* (DDIM).

3.1 Forward vs generation, and the DDPM chain

For diffusion, the key distinction is:

$$0 \to T$$

is the forward noising process, and

$$T \to 0$$

is generation. A standard DDPM has

$$x_0 \to x_1 \to \cdots \to x_T$$

with

$$\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i.$$

3.2 Collapsing Gaussian steps → Spaced DDPM

Many Gaussian forward steps can be collapsed because Gaussians compose. For example,

$$\frac{\alpha_{400}}{\bar{\alpha}_{200}} = \alpha_{201} \cdots \alpha_{400}$$

is exactly the total signal retention from timestep 200 to 400. That lets Spaced DDPM define a new shorter Markov chain using

$$\alpha'_k = \frac{\bar{\alpha}_{t_k}}{\bar{\alpha}_{t_{k-1}}}, \qquad \beta'_k = 1 - \alpha'_k.$$

Then reverse sampling can genuinely jump

$$x_{1000} \to x_{800} \to x_{600} \to \cdots$$

using the Gaussian posterior of that new shortened DDPM and fresh stochastic noise.

3.3 DDIM: a deterministic trajectory between arbitrary noise levels

DDIM gets the same-looking jumps for a different reason. At x_t, predict $\hat{\epsilon}$, infer

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\hat{\epsilon}}{\sqrt{\bar{\alpha}_t}},$$

and then directly place the sample at an earlier noise level s:

$$x_s = \sqrt{\bar{\alpha}_s}\hat{x}_0 + \sqrt{1-\bar{\alpha}_s}\hat{\epsilon}$$

for deterministic DDIM.

3.4 Spaced DDPM vs DDIM, and the modern picture

So both can internally use $\hat{x}_0$, but:

$$\boxed{\text{Spaced DDPM} = \text{shorter stochastic Markov chain}}$$

whereas

$$\boxed{\text{DDIM} = \text{deterministic/controlled trajectory between arbitrary noise}}$$

And the modern broader picture is that the field has increasingly moved toward latent autoencoders + Transformer backbones + diffusion/flow-matching or rectified-flow objectives, with old-style Spaced DDPM being more historically important than central to the newest systems.

> **Key takeaway.** Same jumps, different guarantees. Spaced DDPM builds a genuine shorter Markov chain ($\alpha'_k = \bar{\alpha}_{t_k}/\bar{\alpha}_{t_{k-1}}$) and still injects fresh noise each step. DDIM fixes an $\hat{x}_0$ estimate and re-places it at any noise level deterministically — one roll of the dice at x_T, then a fixed recipe.

Chapter 4: The Diffusion Handbook (Parts I–X)

> ⚠️ **Source not yet recovered — content pending.**
>
> `diffusion-handbook.pdf` is the project's 84-page reference (Parts I–X: the probabilistic core, training/sampling/distillation recipes, inference optimisation, real-time video, world models, evaluation, and a corrections log). It exists **only as a PDF** — a prior session searched the whole project and the home directory and found **no markdown/HTML/TeX source**. It cannot be copied in "word for word" until we either (a) install a PDF text tool (`poppler`) and extract/transcribe it, or (b) locate/re-supply its original source. Until then this chapter is a placeholder so the rest of the notebook is complete and the handbook is not silently dropped. **The PDF itself is untouched and safe.**

Chapter 5: Image conditioning — vision encoder + image projector

Text conditioning answers "what should the model *make*?" with a caption. But a huge amount of practice is conditioning on **another image**: "make it in *this* style," "keep *this* person's face," "use *this* as a reference." This chapter builds the single most common recipe for that — turning a reference image into a handful of conditioning tokens the denoiser can attend to — and shows why it is written the way the seed formula is written:

$$I = P_{\text{img}}\big(E_{\text{vision}}(x_0)\big).$$

5.1 Two kinds of conditioning: semantic vs pixel-aligned

Before any formula, one distinction decides *everything* about how you wire an image in. Ask: **do the conditioning pixels line up, cell-for-cell, with the pixels I am generating?**

- **Pixel-aligned** — inpainting mask, a depth/edge map, a low-res image for super-resolution. Position (i, j) of the condition means the same place as position (i, j) of the output. The tensor's own indexing carries the correspondence, so the right move is to **concatenate on the channel axis** and let the first convolution read both at once. No learning of "where things go" is needed.
- **Semantic / non-aligned** — a style reference, an identity photo, "the vibe of this painting." There is *no* pixel correspondence: the reference face may be centered while the output face is off to the left. Concatenating by position would be meaningless. Instead you compress the reference into a small set of **content tokens** and let **cross-attention** decide, per output location, what to borrow.

The seed formula lives entirely in the **second** world. I is a bag of semantic tokens, not a spatial map.

5.2 Image conditioning as tokens: $I = P_{\text{img}}(E_{\text{vision}}(x_0))$

Read the formula right-to-left — it is a two-stage pipeline:

$$x_0 \xrightarrow{E_{\text{vision}}} f \xrightarrow{P_{\text{img}}} I.$$

- x_0 — the **clean reference image** (not a noisy latent; see §5.3).
- E_{vision} — a **frozen, pretrained vision encoder**, typically a CLIP or DINO image tower. It maps the image to features f. Concretely a CLIP ViT-L/14 gives you either a single pooled embedding ($f \in \mathbb{R}^{1024}$) or the full patch sequence (e.g. $f \in \mathbb{R}^{257\times1024}$ — a `[CLS]` token plus 16×16 patch

tokens). It is frozen because it already knows "what images mean"; retraining it would waste that and risk collapse.

- P_{img} — a **small learned image projector**. Its whole job is to bridge the *domain gap*: E_{vision} lives in CLIP/DINO space, but the denoiser's cross-attention was built to consume **text-encoder** tokens of some width d. P_{img} maps vision features into that same d and, usually, into a fixed small **number** of tokens.
- I — the resulting **image conditioning tokens**, shape $(B,\ M,\ d)$ with M small (IP-Adapter's resampler famously produces $M = 16$ or 64). These are the key/value sequence the denoiser cross-attends to.

P_{img} is not always a bare linear layer. When E_{vision} emits a long patch sequence, P_{img} is often a small **resampler / Perceiver-style attention block** with M learned query tokens:

```
# P_img as a learnable resampler (schematic)
q = learned_queries            # (M, d)      M ≪ 257
f = E_vision(x0)               # (257, 1024) frozen features
I = cross_attn(q, kv=proj(f))  # (M, d)      queries pull a fixed-size
summary
```

So P_{img} does two things at once: **project** (width $1024 \to d$) and **summarize** ($257 \to M$ tokens).

5.3 Why encode the *clean* image, and why a projector at all

Two design choices in the formula are easy to skip past.

Why x_0 and not a noisy x_t? The reference is a *fixed side input*, not part of the trajectory being denoised. We want its full semantic content, so we feed the clean image and encode it **once**, outside the sampling loop. Its tokens I are then reused unchanged at every timestep — cheap, and stable.

Why P_{img} at all — why not feed $E_{\text{vision}}(x_0)$ straight in? Two mismatches make that impossible:

1. **Width.** CLIP features are (say) 1024-d; the cross-attention projections W_K, W_V expect width d (the text-token width). A linear map is the minimum fix.
2. **Semantics/scale.** CLIP-space directions are not text-encoder-space directions, and their scale is different. A tiny trained projector aligns the two distributions so the frozen attention weights actually "understand" the injected tokens.

That is the whole reason the projector is a *learned* module wrapped around a *frozen* encoder: freeze the expensive part that already works, train only the thin adapter that translates it.

5.4 Where I actually enters: decoupled cross-attention

Naively you could concatenate I onto the text tokens and run one cross-attention. In practice the strong recipe (IP-Adapter) keeps them **separate** — *decoupled cross-attention* — adding a second attention that shares the query but has its **own** W'_K, W'_V for the image tokens:

$$\text{out} = \underbrace{\text{Attn}(Q,\, K_{\text{txt}},\, V_{\text{txt}})}_{\text{original, frozen}} \;+\; \lambda\, \underbrace{\text{Attn}(Q,\, K_{\text{img}},\, V_{\text{img}})}_{\text{new, trained}},$$

with

$$K_{\text{img}} = I\, W'_K, \qquad V_{\text{img}} = I\, W'_V.$$

Why decouple? (i) It lets the base text-to-image model stay **frozen** — only W'_K, W'_V (and P_{img}) are trained, so the adapter is tiny and doesn't degrade the original text behavior. (ii) The scalar λ becomes a clean **image-conditioning strength** knob at inference: $\lambda = 0$ recovers the pure text model, larger λ leans harder on the reference. This is exactly the "one vector or many / add a side stream" pattern generalized to a whole image.

5.5 Tokens vs channel-concat: choosing by whether the pixels line up

Tie §5.1 back to the two mechanisms so the decision is mechanical:

Situation	Correspondence	Mechanism	Why
Inpaint / depth / super-res	pixel-aligned	**channel concat** into `conv_in`	indexing *is* the alignment; no cross-attn needed
Style / identity / reference	none	**tokens via** $I = P_{\text{img}}(E_{\text{vision}}(x_0))$ + cross-attn	attention learns *what* to borrow, not *where*

The failure mode of using the wrong one is instructive: channel-concat a style reference and the model tries to copy it *positionally* (ghosting, mis-registered content); tokenize a depth map and you throw away the precise geometry that only survives in the spatial grid. Match the mechanism to the correspondence.

> **Key takeaway.** $I = P_{\text{img}}(E_{\text{vision}}(x_0))$ is the semantic-image conditioning recipe: a **frozen** vision encoder turns the clean reference into features, a **small trained projector/resampler** maps them to M tokens of the denoiser's width, and those tokens enter through a **decoupled** cross-

> attention with its own W'_K, W'_V and a strength scalar λ. Use it only when the reference does **not** align pixel-for-pixel with the output — for aligned conditions, channel-concat and skip the encoder entirely.

Chapter 6: Flow matching and few-step generators — FM → Consistency → FMM → AYF → MeanFlow → iMF

Diffusion sampling (Chapter 3) is slow because generation integrates an ODE/SDE with many network evaluations. This chapter builds the modern line of work that collapses that trajectory into **one or a few steps**, told as a single story: flow matching gives the velocity field; consistency models, flow map matching, and MeanFlow each learn a *different object* that lets you jump across time; and AYF/iMF are the training-stability refinements that made those objects learnable from scratch. The payoff for the rest of the notebook is a differentiable map $\text{noise} \to \text{image}$ with a *short* computational graph — exactly what Chapters 7–8 need for reward guidance.

6.0 One notation for the whole family

We use one convention throughout, which differs from the DDPM convention of Chapter 3 (there $t = 0$ was data, $t = T$ noise — same spirit, continuous here):

$$t = 0 : \text{ data}, \qquad t = 1 : \text{ noise},$$

with $x \sim p_{\text{data}}$ and $\epsilon \sim N(0, I)$. For flow maps,

$$X_{s,t}(x_s) = x_t$$

reads "start from state x_s at time s, return the state at time t."

6.1 Flow matching: a probability path from data to noise

The simplest straight-line interpolation between a data point and noise is

$$x_t = (1 - t)\, x + t\, \epsilon.$$

At the endpoints $x_0 = x$ (data) and $x_1 = \epsilon$ (noise). For a *fixed* clean x, the interpolant is Gaussian:

$$x_t \mid x \sim N\big((1-t)x,\ t^2 I\big),$$

so the marginal at time t is a mixture of all those Gaussian clouds:

$$p_t(y) = \int p_{\text{data}}(x)\, N\big(y; (1-t)x,\ t^2 I\big)\, dx.$$

This Gaussian conditional-path construction is the general flow-matching framework; different schedules recover diffusion-style paths as special cases.

6.2 Conditional velocity vs marginal velocity

Differentiate one sampled path:

$$\dot{x}_t = \frac{dx_t}{dt} = \epsilon - x \;=:\; v_c,$$

the **conditional (per-sample) velocity**. It is *not* the deterministic marginal field, because a single visible state $x_t = y$ can arise from many hidden pairs (x, ϵ), each with a different $\epsilon - x$. The deterministic velocity attached to the visible state must therefore average them:

$$\boxed{v(y,t) = \mathbb{E}[\epsilon - x \mid x_t = y]}.$$

6.3 Why conditional flow matching works (the MSE argument)

We would like to regress onto the marginal field,

$$L_{\mathrm{FM}} = \mathbb{E}\big\| v_\theta(x_t,t) - v(x_t,t) \big\|^2,$$

but $v(x_t,t)$ contains an inaccessible conditional expectation. Instead we regress onto the *easy* per-sample target:

$$L_{\mathrm{CFM}} = \mathbb{E}\big\| v_\theta(x_t,t) - (\epsilon - x) \big\|^2.$$

The basic MSE theorem $\arg\min_f \mathbb{E}\|f(X) - Y\|^2 = \mathbb{E}[Y \mid X]$ gives

$$v^*_\theta(y,t) = \mathbb{E}[\epsilon - x \mid x_t = y] = v(y,t).$$

So minimizing the conditional loss recovers the marginal field; the two losses even share the same gradient in θ. This is not merely a trick — the next section shows the field it recovers genuinely transports p_t.

6.4 The continuity equation: why the ODE transports p_t

Take any smooth test function $\varphi : \mathbb{R}^D \to \mathbb{R}$. Along interpolant trajectories,

$$\frac{d}{dt}\varphi(x_t) = \nabla\varphi(x_t)^\top \dot{x}_t \;\Rightarrow\; \frac{d}{dt}\mathbb{E}[\varphi(x_t)] = \mathbb{E}\big[\nabla\varphi(x_t)^\top \dot{x}_t\big].$$

Apply the tower rule and the definition $v(x_t,t) = \mathbb{E}[\dot{x}_t \mid x_t]$:

$$\frac{d}{dt}\mathbb{E}[\varphi(x_t)] = \mathbb{E}\big[\nabla\varphi(x_t)^\top v(x_t,t)\big] = \int \nabla\varphi(y)^\top v(y,t)\, p_t(y)\, dy.$$

But also $\frac{d}{dt}\mathbb{E}[\varphi(x_t)] = \int \varphi(y)\, \partial_t p_t(y)\, dy$. Integration by parts on the right, $\int \nabla\varphi^\top (p_t v) = -\int \varphi\, \nabla\cdot(p_t v)$, gives for arbitrary φ:

$$\boxed{\partial_t p_t + \nabla \cdot (p_t v) = 0}.$$

That is the continuity equation, so the deterministic ODE $\dot{x}_t = v(x_t, t)$ transports the same marginals p_t.

6.5 Straight interpolant trajectory vs probability-flow trajectory

This distinction is essential. The sampled straight line has velocity $\epsilon - x$; the probability-flow ODE has velocity $\mathbb{E}[\epsilon - x \mid x_t]$. In general

$$\epsilon - x \;\neq\; v(x_t, t).$$

So the two share the same *marginal distribution* at every t but *not* the same individual trajectory: the sampled interpolants are straight, while the marginal ODE trajectories are curved. That curvature is exactly why single-step Euler is inaccurate and why few-step methods exist.

6.6 Sampling, and the motivation for few-step methods

With v_θ trained, draw $x_1 \sim N(0, I)$ and integrate *backward* $1 \to 0$:

$$x_0 = x_1 - \int_0^1 v_\theta(x_\tau, \tau)\, d\tau, \qquad x_{t-\Delta t} \approx x_t - \Delta t\, v_\theta(x_t, t).$$

If the trajectory curves, many evaluations are needed. Everything below is an attempt to do this in one or a few network calls.

6.7 Consistency models: predict the endpoint directly

Instead of local velocity, predict the clean endpoint:

$$f_\theta(x_t, t) \approx x_0.$$

The **consistency condition** is that all points on one trajectory map to the same endpoint: $f(x_s, s) = f(x_t, t) = x_0$. The boundary condition $f(x_0, 0) = x_0$ is baked in architecturally with skip/output coefficients,

$$f_\theta(x_t, t) = c_{\text{skip}}(t)\, x_t + c_{\text{out}}(t)\, F_\theta(x_t, t), \qquad c_{\text{skip}}(0) = 1,\; c_{\text{out}}(0) = 0,$$

so $f_\theta(x_0, 0) = x_0$ automatically.

6.8 Consistency distillation and consistency training

Distillation uses a pretrained teacher ODE. Pick adjacent times $t_{n+1} > t_n$, take a noisy $x_{t_{n+1}}$, run one small teacher step $x_{t_{n+1}} \to \hat{x}_{t_n}$, and train the two to agree on the endpoint:

$$L_{\text{CD}} = d\big(f_\theta(x_{t_{n+1}}, t_{n+1}),\ f_{\theta^-}(\hat{x}_{t_n}, t_n)\big),$$

where θ^- is usually an EMA target. **Training from scratch** (no teacher) imposes the same condition $f_\theta(x_t, t) \approx f_{\theta^-}(x_s, s)$ but must construct good same-trajectory pairs from the forward corruption process alone — fragile, hence the carefully designed discretization curricula. One-step sampling is $x_0 = f_\theta(x_1, 1)$; multistep re-noises and reapplies, which can reintroduce endpoint error.

6.9 Flow map matching: the two-time solution operator

Consistency models only learn a map to *one* distinguished endpoint. Flow map matching (FMM) generalizes to arbitrary start/destination times:

$$X_\theta(x_s, s, t) \approx X_{s,t}(x_s) = x_t.$$

The exact map satisfies $X_{s,s}(x) = x$, enforced by the parameterization

$$X_\theta(x, s, t) = x + (t - s)\, F_\theta(x, s, t) \ \Rightarrow\ X_\theta(x, s, s) = x.$$

FMM admits both teacher-distillation and direct-from-scratch training, and lets you pick the step count *after* training.

6.10 The Lagrangian identity and FMM-LMD

Fix the start (x_s, s) and vary the destination t. Then $y(t) = X_{s,t}(x_s)$ is the ODE trajectory itself, so

$$\boxed{\partial_t X_{s,t}(x_s) = v\big(X_{s,t}(x_s), t\big)}.$$

With a pretrained teacher v_ϕ, the **Lagrangian map distillation** residual and loss are

$$r_{\text{LMD}} = \partial_t X_\theta(x_s, s, t) - v_\phi\big(X_\theta(x_s, s, t), t\big), \qquad L_{\text{FMM-LMD}} = \mathbb{E}\|r_{\text{LMD}}\|^2$$

Procedure: sample (x, ϵ, s, t), form $x_s = I_s$, push through X_θ, compute $\partial_t X_\theta$ and the teacher velocity at the predicted destination, and MSE them.

6.11 The Eulerian identity and FMM-EMD

Now fix the destination t and slide the *start* along one teacher trajectory. Define $g(\tau) = X_{\tau,t}(x_\tau)$. Every start lies on the same trajectory with the same destination, so $g(\tau) = x_t$ is constant, $dg/d\tau = 0$. The chain rule $\partial_s X + J_x X\, \dot{x}_\tau = 0$ with $\dot{x}_\tau = v(x_\tau, \tau)$ gives the **Eulerian identity**

$$\boxed{\partial_s X + J_x X\, v = 0}.$$

The **Eulerian map distillation** residual and loss:

$$r_{\text{EMD}} = \partial_s X_\theta + J_x X_\theta \, v_s, \qquad L_{\text{FMM-EMD}} = \mathbb{E}\|r_{\text{EMD}}\|^2.$$

6.12 The JVP tangent $(v_s, 1, 0)$, and why it is expensive

The network input is $u = (x_s, s, t)$. To move the start along the teacher trajectory, perturb $x_s(\delta) = x_s + \delta v_s$, $s(\delta) = s + \delta$, $t(\delta) = t$ — tangent $d = (v_s, 1, 0)$. A single **Jacobian-vector product** delivers the Eulerian residual:

$$J_u X_\theta \, (v_s, 1, 0) = J_x X_\theta \, v_s + \partial_s X_\theta = r_{\text{EMD}}.$$

The catch for FMM: the loss $\|r_\theta\|^2$ has gradient $\nabla_\theta L = 2(\partial r_\theta/\partial\theta)^\top r_\theta$, and r_θ *already* contains first derivatives of X_θ. So backprop needs mixed second derivatives $\partial^2 X/\partial\theta\partial x$, $\partial^2 X/\partial\theta\partial s$ (and $\partial^2 X/\partial\theta\partial t$ for LMD). The JVP gets the first derivative cheaply, but differentiating through the JVP is still higher-order — the cost AYF is designed to avoid.

6.13 Direct FMM from scratch: cycle + dynamics

With no teacher, use the stochastic interpolant I_t and its sample velocity $\dot{I}_t$. Naïvely training $\partial_t X(I_s, s, t) \approx \dot{I}_t$ fails: it learns the conditional-mean/denoising map $\mathbb{E}[I_t \mid I_s]$, not the deterministic probability-flow map. The fix pairs a **cycle** constraint with a **dynamics** constraint. Map back then forward,

$$y_s = X_\theta(I_t, t, s), \qquad \hat{I}_t = X_\theta(y_s, s, t),$$

and combine

$$L_{\text{cycle}} = \|\hat{I}_t - I_t\|^2, \quad L_{\text{dyn}} = \|\partial_t X_\theta(y_s, s, t) - \dot{I}_t\|^2, \quad L_{\text{direct}} = \mathbb{E}[L_{\text{cycle}}$$

The cycle enforces forward/backward trajectory correspondence; the dynamics term forces the correct marginal velocity in expectation.

6.14 AYF: stop-gradient the residual, use it as a direction

Align Your Flow (AYF) keeps the same flow-map geometry but changes the optimization: **measure the differential residual, stop-gradient it, and use it as a first-order training direction** — avoiding backprop through the JVP.

AYF-EMD. Let $A = X_\theta(x_s, s, t)$ and take a tiny teacher step at the start, $s' = s + \delta$, $x_{s'} = x_s + \delta v_s$, giving the shorter map $B = X_{\theta^-}(x_{s'}, s', t)$. The exact map has $A = B$, and Taylor expansion gives $B - A = \delta \, r_{\text{EMD}} + O(\delta^2)$. The one-sided loss $L_\delta = \frac{1}{2}\|A - \text{sg}(B)\|^2$ has gradient $\nabla_\theta L_\delta \approx -\delta \, (\partial A/\partial\theta)^\top r_{\text{EMD}}$ — needing the *numerical* r_{EMD} (one JVP, detached), not $\partial r_{\text{EMD}}/\partial\theta$.

AYF-LMD. Let $A = X(x_s, s, t)$ and $B = X(x_s, s, t - \delta) = A - \delta\partial_t X$. Take the remaining tiny teacher step $T = B + \delta v_\phi(B, t - \delta)$; smoothness gives $A - T = \delta \, r_{\text{LMD}} + O(\delta^2)$, and the one-sided loss has gradient $\nabla_\theta L \approx$

$\delta\,(\partial A/\partial\theta)^\top r_{\text{LMD}}$.

So FMM vs AYF is the same fixed point $r = 0$, different optimization:

$$\text{FMM: } \nabla_\theta \|r\|^2 = J_\theta r^\top r \qquad \text{vs} \qquad \text{AYF: } r^- = \text{sg}(r),\ \ \nabla_\theta L \sim J_\theta X^\top r^-.$$

6.15 MeanFlow: learn the average velocity

MeanFlow learns neither local velocity nor an endpoint, but the **average velocity over an interval**:

$$u(z_t, r, t) = \frac{1}{t-r}\int_r^t v(z_\tau, \tau)\, d\tau.$$

Because $z_t - z_r = \int_r^t v\, d\tau$, one average-velocity prediction jumps the whole interval:

$$z_r = z_t - (t-r)\,u(z_t, r, t), \qquad \text{one-step: } z_0 = z_1 - u(z_1, 0, 1).$$

6.16 The MeanFlow identity and its JVP target

Multiply out $(t-r)u = \int_r^t v\, d\tau$ and differentiate in t (holding r). The fundamental theorem gives v on the right; the product rule gives $u + (t-r)\,du/dt$ on the left:

$$\boxed{v = u + (t-r)\frac{du}{dt}}, \qquad \text{i.e.} \qquad u = v - (t-r)\frac{du}{dt}.$$

The total derivative along the trajectory is $\frac{du}{dt} = J_z u\, v + \partial_t u$, so the JVP tangent is $(v, 0, 1)$. Since the true v is unavailable, **original MeanFlow** substitutes the conditional $v_c = \epsilon - x$, computes $\dot{u}_\theta = \text{JVP}(u_\theta; (v_c, 0, 1))$, forms the target $u_{\text{tgt}} = v_c - (t-r)\dot{u}_\theta$, and trains

$$L_{\text{MF}} = \mathbb{E}\|u_\theta - \text{sg}(u_{\text{tgt}})\|^2.$$

The stop-gradient prevents second-order differentiation through the JVP. Note the exact bridge to FMM: $X_{t,r}(z_t) = z_t - (t-r)u(z_t, r, t)$ — average velocity is just displacement normalized by interval length.

6.17 iMF: a network-independent, guidance-flexible target

Improved MeanFlow (iMF) fixes two problems.

Problem 1 — the target depends on the network. u_{tgt} contains u_θ (via its JVP), so you train u_θ against a moving, self-referential target. Worse, the JVP uses the high-variance conditional velocity v_c where theory wants the marginal v, and the Jacobian magnifies that variance.

Fix, step 1 — read the identity as a v-loss. Rearranging $v = u + (t-r)\,du/dt$, define the compound predictor

$$V_\theta = u_\theta + (t-r)\,\mathrm{sg}\left[\tfrac{du_\theta}{dt}\right], \qquad L = \mathbb{E}\|V_\theta - v_c\|^2.$$

This is MeanFlow viewed backwards: a flow-matching v-loss reparameterized by u_θ.

Fix, step 2 — use a marginal-velocity estimate as the JVP tangent. The tangent should be $(v, 0, 1)$, not $(v_c, 0, 1)$. iMF estimates v from the model itself by collapsing the interval,

$$v_\theta(z_t, t) = u_\theta(z_t, t, t) \quad \text{since} \quad \lim_{r \to t} u(z_t, r, t) = v(z_t, t)$$

(or an auxiliary velocity head). Then $\dot{u}_\theta = \mathrm{JVP}(u_\theta; (v_\theta, 0, 1))$, $V_\theta = u_\theta + (t-r)\,\mathrm{sg}(\dot{u}_\theta)$, and

$$L_{\text{iMF}} = \mathbb{E}\|V_\theta - (\epsilon - x)\|^2.$$

Now the *prediction* depends only on the model's own marginal-velocity estimate, while the conditional $\epsilon - x$ stays on the right where MSE marginalizes it — much more stable training.

Fix 2 — guidance flexibility. Original MeanFlow can bake CFG into the average velocity (keeping one NFE) but must fix the CFG scale at *training* time. iMF instead conditions the model, $u_\theta(z_t, r, t, c, \omega, \dots)$, on the guidance strength ω (and optionally a CFG time interval), so one trained one-step model can choose ω at inference.

6.18 The unifying view: what object does the network learn?

Every method in this chapter answers one question differently:

$$\text{FM: } v(x_t, t) = \text{instantaneous velocity}$$

$$\text{Consistency: } f(x_t, t) = \text{endpoint}$$

$$\text{FMM: } X(x_s, s, t) = \text{arbitrary finite-time destination}$$

$$\text{MeanFlow: } u(x_t, r, t) = \text{average velocity over an interval}$$

and the last two are tied by $X_{t,r}(x_t) = x_t - (t-r)\,u(x_t, r, t)$. FMM/AYF ask how to learn the *map* robustly; MeanFlow/iMF ask how to learn its *normalized displacement* robustly. All of them ultimately hand you a differentiable $\text{noise} \to \text{image}$ map — the object Chapters 7–8 exploit.

> **Key takeaway.** Flow matching learns the marginal velocity $v = \mathbb{E}[\epsilon - x \mid x_t]$ by regressing on the cheap conditional target $\epsilon - x$ (MSE recovers the conditional mean; the continuity equation guarantees it transports p_t). The sampled interpolants are *straight* but the probability-flow trajectory is

curved, which is why one Euler step is wrong and few-step methods exist. Consistency models learn the endpoint, FMM the two-time map $X_{s,t}$, MeanFlow the average velocity u (with $X_{t,r} = x_t - (t-r)u$). The training-stability story is the stop-gradient: FMM minimizes $\|r\|^2$ (needs higher-order backprop through a JVP), while AYF and MeanFlow/iMF detach the residual and use it as a first-order direction — and iMF further replaces the high-variance v_c in the JVP with the model's own marginal estimate $v_\theta = u_\theta(z_t, t, t)$.

Chapter 7: Guidance — tilting a generative prior toward a reward

Chapters 1–6 build generators that sample from a learned prior $p_{\text{gen}}(x)$. This chapter asks the orthogonal question: given a *frozen* prior and a reward $R(x)$ (a classifier logit, an aesthetic score, later a brain-activation predictor), how do we bias generation toward high reward **without destroying the naturalness the prior provides**? The whole chapter is organized around one target distribution and the many ways to approximate it — from training-free gradient guidance to fine-tuning. It is the bridge from "how to generate" to the fMRI application in Chapter 8.

7.1 The distribution-level goal: reward minus KL

The principled objective trades reward against staying close to the prior:

$$\max_{q} \; \mathbb{E}_{x\sim q}[R(x)] - \beta \,\mathrm{KL}\big(q \,\|\, p_{\text{gen}}\big).$$

The first term wants high reward; the KL term says *do not drift away from the natural images the generator produces*. The temperature β sets the trade-off.

7.2 The optimal tilted distribution $q^* \propto p_{\text{gen}} e^{R/\beta}$

Expand the objective and add a normalization multiplier λ:

$$F = \int q(x) R(x)\, dx - \beta \int q(x) \log \frac{q(x)}{p_{\text{gen}}(x)}\, dx + \lambda\Big(\int q(x)\, dx - 1\Big).$$

The functional derivative is

$$\frac{\delta F}{\delta q(x)} = R(x) - \beta\Big[\log \frac{q(x)}{p_{\text{gen}}(x)} + 1\Big] + \lambda.$$

Setting it to zero gives $\log \frac{q(x)}{p_{\text{gen}}(x)} = \frac{R(x)}{\beta} + \text{const}$, hence the **exponentially tilted** (Gibbs) distribution

$$\boxed{q^*(x) \propto p_{\text{gen}}(x)\, e^{R(x)/\beta}}.$$

Every method below is an approximate sampler or optimizer for this object: reweight the prior by $e^{R/\beta}$ — strongly if β is small, gently if β is large.

7.3 Classifier guidance and CFG

For conditional generation, classifier guidance adds the gradient of a classifier log-likelihood. By Bayes, $\log p(x_t \mid c) = \log p(x_t) + \log p(c \mid x_t) + C$, so

$$\nabla_x \log p(x_t \mid c) = \nabla_x \log p(x_t) + \nabla_x \log p(c \mid x_t).$$

Classifier-free guidance (CFG) avoids a separate classifier by combining the model's conditional and unconditional predictions, in velocity form:

$$v_{\text{CFG}} = v_{\text{uncond}} + \omega \left(v_{\text{cond}} - v_{\text{uncond}}\right).$$

Pros: strong semantic control, no external differentiable reward. *Cons:* usually two network evaluations per step (unless distilled/amortized — cf. iMF §6.17), and large ω lowers diversity.

7.4 Universal guidance: reward on the predicted clean image

For an *arbitrary* differentiable reward R, guide at each diffusion step through the model's predicted clean image $\hat{x}_0(x_t)$:

$$g_t = \nabla_{x_t} R\big(\hat{x}_0(x_t)\big) = J_{\hat{x}_0}(x_t)^\top \nabla_{\hat{x}_0} R,$$

and nudge the denoising direction: $\text{base direction} + \lambda_t\, g_t$. Universal Guidance shows this works for any differentiable guidance function with no retraining. *Pros:* training-free, arbitrary targets. *Cons:* an extra backward pass through reward + $\hat{x}_0$ at many steps; strong gradients can exploit the reward network.

7.5 Manifold-preserving guidance (MPGD)

Naïve pixel-space gradient ascent pushes images **off the natural-image manifold**: $R(x)$ goes up but the image looks wrong (reward hacking). MPGD constrains guidance to directions the generator's geometry can represent. If a decoder defines the manifold $x = D(z)$, apply reward gradients through the latent:

$$\nabla_z R\big(D(z)\big) = J_D(z)^\top \nabla_x R,$$

rather than arbitrary pixel directions. *Pros:* less off-manifold drift, more naturalistic samples. *Cons:* the manifold caps what guidance can achieve, and results depend on the autoencoder's quality.

7.6 Optimizing the starting noise

For a frozen generator $x = G(\epsilon)$, search for noise that yields high reward:

$$\max_{\epsilon}\; R\big(G(\epsilon)\big) - \tfrac{\lambda}{2}\|\epsilon\|^2, \qquad \nabla_\epsilon R = J_G(\epsilon)^\top \nabla_x R.$$

Pros: no fine-tuning; optimizes the true terminal reward. *Cons:* every step differentiates through the whole generator; local minima; ϵ can drift to atypical regions; reward hacking.

7.7 Fine-tuning the generator (AlignProp)

Amortize the search by training the generator itself:

$$\max_{\theta} \; \mathbb{E}_{\epsilon}\big[R(G_\theta(\epsilon))\big] - \beta\, D\big(p_\theta \,\|\, p_{\text{base}}\big), \qquad \nabla_\theta R = J_\theta G_\theta^\top \nabla_x R.$$

AlignProp backpropagates reward through diffusion sampling to fine-tune text-to-image models. *Pros:* amortized — every future sample benefits. *Cons:* expensive, prone to reward hacking and mode collapse; needs strong regularization.

7.8 Flow-map terminal guidance: a short backprop graph

This is where Chapter 6 pays off. With a one/few-step flow map $x_0 = X_\theta(x_t, t, 0)$, the reward gradient is one backward pass through a *short* graph:

$$\nabla_{x_t} R\big(X_\theta(x_t, t, 0)\big) = J_{x_t} X_\theta^\top \, \nabla_{x_0} R.$$

Compared with differentiating through 50 diffusion steps, the graph is dramatically shorter, so optimizing x_t or the initial noise x_1 against a *terminal* reward is far cheaper. *Pros:* true terminal reward, shallow backprop, fast many-seed search. *Cons:* quality is bounded by the accuracy of the one/few-step map; its natural-image distribution may be narrower than a strong many-step model's.

> **Key takeaway.** All reward guidance approximates the tilted distribution $q^*(x) \propto p_{\text{gen}}(x)\, e^{R(x)/\beta}$ — reweight the prior by the reward, with β setting how hard. The methods trade off *where* the optimization lives and *how long* the backprop graph is: CFG/classifier guidance (built-in, cheap, semantic only); Universal Guidance (training-free, per-step reward backprop through $\hat{x}_0$); MPGD (constrain to the manifold to resist reward hacking); noise optimization and AlignProp (per-sample vs amortized, both differentiate the full generator); and flow-map terminal guidance, which uses a one-step map to make $\nabla_{\text{noise}} R$ a single short backward pass. The recurring enemy is **reward hacking** — a high score that reflects an off-manifold exploit of the reward model, not a genuinely good image.

Chapter 8: Application — maximizing fMRI ROI activation with generative priors

This chapter instantiates the guidance machinery of Chapter 7 for a concrete neuroscience goal: **synthesize images that drive a target brain region**. The reward is a differentiable fMRI *encoding model* that predicts voxel responses from an image; the generative prior keeps those images natural. We work through the whole method ladder — training-free per-step guidance (BrainDiVE), specificity objectives, flow-map terminal optimization, and a distribution-level formulation — and end on the point that matters most in practice: the real adversary is not the sampler but **reward hacking of the encoder**.

8.1 The ROI reward from a differentiable brain encoder

Let a differentiable encoder predict V voxel responses from image x:

$$E_{\text{brain}}(x) = \hat{r}(x) \in \mathbb{R}^V.$$

For a target ROI $T \subset \{1, \dots, V\}$ the basic reward is the mean predicted response over the ROI,

$$A_T(x) = \frac{1}{|T|} \sum_{v \in T} \hat{r}_v(x), \qquad R(x) = A_T(x).$$

BrainDiVE is exactly this idea: a voxel-wise fMRI encoder (OpenCLIP-based) on top of latent diffusion (Stable Diffusion), averaging ROI activations and backpropagating their gradient through the predicted image into the diffusion process.

8.2 Method A — BrainDiVE-style step-by-step diffusion guidance

This is Universal Guidance (§7.4) with $R = A_T$. At latent z_t, form the predicted clean image $\hat{x}_0(z_t)$, run the encoder, and chain the gradient back:

$$g_t = \nabla_{z_t} A_T\big(\hat{x}_0(z_t)\big) = J_{\hat{x}_0}(z_t)^\top \, J_{E_{\text{brain}}}(\hat{x}_0)^\top \, \nabla_{\hat{r}} A_T,$$

then modify the diffusion step with g_t. *Pros:* generator frozen, strong natural-image prior, trivial to swap ROI. *Cons:* reward + backprop at many steps (slow); early noisy $\hat{x}_0$ give imperfect gradients; the encoder can be exploited.

8.3 Method B — specificity: "X but not Y"

Maximizing A_T alone can produce globally hyper-activating images that light up *everything*. For interpretable, **selective** results, subtract a control:

$$R(x) = A_T(x) - \lambda\, A_C(x),$$

e.g. "food ROI but not face ROI": $R = A_{\text{food}} - \lambda A_{\text{face}}$. Variants compare the target to all non-target regions or to the hardest competitor:

$$R(x) = A_T(x) - \lambda\, \frac{1}{K}\sum_{k \neq T} A_k(x), \qquad R(x) = A_T(x) - \lambda\, \max_{k \neq T} A_k(x).$$

This yields *selectivity*, not generic high activation — the scientifically meaningful quantity.

8.4 Normalizing voxel responses

Voxels differ in scale, so raw means are dominated by high-variance voxels. Standardize first,

$$z_v(x) = \frac{\hat{r}_v(x) - \mu_v}{\sigma_v}, \qquad A_T(x) = \frac{1}{|T|}\sum_{v \in T} z_v(x),$$

and optionally weight by encoding reliability w_v (e.g. held-out encoding correlation):

$$A_T(x) = \frac{\sum_{v \in T} w_v\, z_v(x)}{\sum_{v \in T} w_v}.$$

8.5 Methods C–F — manifold guidance, noise optimization, flow-map, LoRA

These transplant §7.5–7.8 directly, trading cost against robustness:

- **C — Manifold-preserving ROI guidance (cf. §7.5).** Optimize through the generator latent, $\nabla_z A_T(D(z)) = J_D(z)^\top \nabla_x A_T$, instead of raw pixels. Less encoder-adversarial nonsense, more naturalistic images; but may suppress genuine-but-unusual high-activation stimuli.
- **D — Optimize initial noise (cf. §7.6).** Freeze G and solve $\epsilon^* = \arg\max_\epsilon [A_T(G(\epsilon)) - \alpha A_C(G(\epsilon)) - \lambda\|\epsilon\|^2]$ with $\nabla_\epsilon R = J_G^\top J_E^\top \nabla_{\hat{r}} R$; many random starts give diverse optima. Exact terminal reward, frozen generator, but expensive per iteration and prone to overfitting the encoder.
- **E — Flow-map / MeanFlow ROI optimization (cf. §7.8).** With a one-step map $x_0 = X_\theta(\epsilon, 1, 0)$ or MeanFlow $x_0 = \epsilon - u_\theta(\epsilon, 0, 1)$, $\nabla_\epsilon R = J_\epsilon X_\theta^\top \nabla_{x_0} A_T$ is one short backward pass — ideal for fast many-seed discovery. Quality is bounded by the one-step map's fidelity.

- **F — Fine-tune a LoRA for an ROI (cf. §7.7).** Train adapters with $L(\theta) = -\mathbb{E}_\epsilon[A_T(G_\theta(\epsilon))] + \beta L_{\text{prior}} + \gamma L_{\text{diversity}}$ (or the specificity form). Cheap generation afterward, but strong reward-hacking risk and must be re-trained per objective unless conditioned.

8.6 Method G — optimize a distribution, not a single image

The most principled scientific formulation is the tilted distribution of §7.2 with $R = A_T$ (or its specificity form):

$$q_T^*(x) \propto p_{\text{natural}}(x)\, \exp\left[\frac{A_T(x) - \lambda A_C(x)}{\beta}\right].$$

The message: *don't ask for one maximally activating image; ask for a natural-image distribution tilted toward selective ROI activation.* Single-image activation maximization is notorious for producing adversarial, unnatural stimuli; the distribution view keeps the prior in control.

8.7 The real adversary: encoder reward hacking

This may matter more than the choice of sampler. An encoder E_{brain} trained on NSD images can output *falsely huge* activations off its training manifold — so a high predicted reward does **not** imply genuine cortical activation. A robust objective therefore combines selectivity, an OOD/naturalness penalty, and a diversity term:

$$R(x) = A_T(x) - \lambda_{\text{control}} A_C(x) - \lambda_{\text{OOD}}\, D_{\text{natural}}(x) + \lambda_{\text{div}}\, D_{\text{diversity}}(x).$$

Crucially, **validation must be independent of the optimization encoder**: optimize with E_1, validate with a separately trained E_2, and ideally confirm with real human fMRI. BrainDiVE itself validates generated preferences with independent semantic/human analyses rather than trusting the encoder score alone.

8.8 A three-level plan, and the master gradient

A sensible progression for this project:

1. **Baseline — reproduce BrainDiVE-style per-step guidance (Method A).** Confirms the ROI encoder and reward work under an already-published procedure.
2. **Better optimization — flow-map / MeanFlow terminal search (Method E):** $\epsilon^* = \arg\max_\epsilon R\big(X_{1,0}(\epsilon)\big)$. Tests whether a one-step map gives better terminal-reward gradients than per-step guidance.
3. **Scientific version — selective, regularized, distribution-level reward (Methods B/G):** $R(x) = A_T(x) - \lambda A_{\text{controls}}(x) - \gamma D_{\text{OOD}}(x)$,

evaluated across independent encoders and, if possible, a new fMRI experiment.

Every method here is one differentiable chain $\textbf{noise} \to \textbf{natural image} \to \textbf{brain activation}$, so the master gradient is always

$$\boxed{\nabla_{\text{noise}} A_T = J_{\text{generator}}^{\top} \, J_{\text{brain}}^{\top} \, \nabla_{\text{ROI}} A_T}.$$

The methodological battle — the whole arc of Chapters 6–8 — is about making that **generator Jacobian** accurate, cheap (few-step, §6), natural-image-preserving (manifold/KL, §7), and resistant to reward hacking (§8.7).

> **Key takeaway.** Driving a brain ROI is reward guidance (Chapter 7) with $R(x) = A_T(x)$ from a differentiable fMRI encoder, under a natural-image prior (Chapter 6). The reward must be **selective** ($A_T - \lambda A_C$), **normalized** across voxels, and **regularized** against OOD exploitation — because the real failure mode is the encoder reporting huge activations for unnatural images, not the sampler. Few-step flow maps (§6) make the terminal gradient $\nabla_\epsilon R = J_\epsilon X_\theta^{\top} \nabla_{x_0} A_T$ a single short backward pass, so the whole stack reduces to one chain $\nabla_{\text{noise}} A_T = J_{\text{generator}}^{\top} J_{\text{brain}}^{\top} \nabla_{\text{ROI}} A_T$; always validate with an independent encoder (ideally real fMRI), never the one you optimized against.

Chapter 9: MeanFlow and iMF in depth — moving targets, JVP noise, and guidance intervals

Chapter 6 gave the *shape* of MeanFlow and iMF: learn the average velocity u, and let iMF reverse the MeanFlow identity into a v-loss. This chapter opens the hood on **why** that reversal matters, and on the guidance machinery that makes a one-step model controllable. Three threads run through it: (1) stop-gradient freezes the target *within* an update but not *across* updates, so original MeanFlow regresses toward a **moving target**; (2) the conditional label v_c is tolerable as a regression *label* but harmful as a JVP *tangent*, which the Jacobian's singular directions can amplify; and (3) a one-step model can still honor a continuous guidance schedule $(\omega, t_{\min}, t_{\max})$ because it learns the *integral* of a virtual guided flow, not the flow itself. All of it reuses the notation of Chapter 6: $z_t = (1-t)x + t\epsilon$, $v_c = \epsilon - x$, $\Delta = t - r$.

9.1 The moving-target problem: stop-gradient ≠ fixed target

Original MeanFlow trains u_θ toward $u_{\text{tgt}} = v_c - \Delta\, \dot{u}_\theta$ with a stop-gradient on the target. Stop-gradient does exactly one thing: it blocks differentiation *through* the target during a single optimizer step, so you never backprop through $J_z u_\theta$ (which would need the second derivative $\partial^2 u_\theta / \partial\theta\, \partial z$). But it does **not** freeze the target across steps. At step k the target is

$$u_{\text{tgt}}^{(k)} = v_c - \Delta\, \dot{u}_{\theta_k},$$

and after the update $\theta_k \to \theta_{k+1}$ you recompute $u_{\text{tgt}}^{(k+1)} = v_c - \Delta\, \dot{u}_{\theta_{k+1}}$. So u_θ is chasing a target whose conditional mean $\mathbb{E}[v_c - \Delta \dot{u}_{\theta_k} \mid z_t, r, t]$ *itself drifts as* θ_k *drifts* — exactly the bootstrapped teacher/student situation of RL value targets. This moving-target instability is iMF's explicit motivation for the v-loss reformulation. Contrast the population optima:

$$\text{iMF: } V_\theta \to \mathbb{E}[v_c \mid z_t, t] = v(z_t, t) \quad \text{(fixed, data-defined)};$$

$$\text{MF: } u_\theta \to \mathbb{E}[v_c - \Delta \dot{u}_{\theta_k} \mid z_t, r, t] \quad \text{(moves with } \theta_k\text{)}.$$

9.2 Conditional and unconditional FM share the label $\epsilon - x$

A point worth stating plainly: the *raw* label does not change between conditional and unconditional training. With class/context c, feeding the network (z_t, t, c) makes the MSE optimum

$$\mathbb{E}[v_c \mid z_t, t, c] = v_{\text{cond}}(z_t, t, c),$$

and feeding $(z_t, t, \varnothing)$ makes it $\mathbb{E}[v_c \mid z_t, t] = v_{\text{uncond}}(z_t, t)$. The algebraic target is $v_c = \epsilon - x$ either way; what changes is *what the predictor is conditioned on*. This is the hook that lets a single network supply both fields for CFG (§9.8).

9.3 Second derivatives are not inherently noisy

It is tempting to justify stop-gradient by "second derivatives are noisy," but there is no such general law. For $f(z) = z^2$, $f'(z) = 2z$ and $f''(z) = 2$ — the second derivative is perfectly stable. The real issue is *sensitivity*: higher derivatives expose higher-order sensitivity. For $f(z) = \sin(100z)$,

$$f'(z) = 100\cos(100z), \qquad f''(z) = -10^4 \sin(100z),$$

so a tiny change in z swings the gradient enormously. A second derivative measures *how quickly the first derivative itself changes*. Large networks mix flat and sharply curved directions, so avoiding unnecessary higher-order gradients is attractive for cost and numerics — but the headline reason to prefer iMF is the moving target (§9.1) and the tangent noise (§9.4), not a blanket "curvature is noisy" claim.

9.4 Noisy v_c *inside* the JVP: $J\eta$ and its covariance

Decompose the conditional label into the true field plus zero-mean noise:

$$v_c = v + \eta, \qquad \mathbb{E}[\eta \mid z] = 0.$$

Original MeanFlow inserts this *noisy sample* into the JVP tangent:

$$J_z u_\theta \, v_c = J_z u_\theta \, v + J_z u_\theta \, \eta.$$

The stray term $J\eta$ still has zero conditional mean (if J is fixed given z), but its covariance is reshaped by the Jacobian:

$$\mathrm{Cov}(J\eta) = J \, \Sigma_\eta \, J^\top.$$

So the Jacobian can *amplify* some noise components and *suppress* others. To see exactly which, read off its singular directions.

9.5 The Jacobian SVD: which directions amplify noise

Take the SVD of $J : \mathbb{R}^n \to \mathbb{R}^m$,

$$J = U \Sigma V^\top,$$

where the columns $v_1, \dots, v_n$ of V are an orthonormal basis of the **input** space, the columns $u_1, \dots, u_m$ of U an orthonormal basis of the **output** space, and there are only $\min(m, n)$ singular values, at most $\mathrm{rank}(J)$ nonzero.

(These u_i, v_i are SVD notation, unrelated to MeanFlow's u, v.) Each right-singular vector maps to its partner:

$$Jv_i = U\Sigma V^\top v_i = U\Sigma e_i = \sigma_i u_i, \qquad \text{i.e. } v_i \xrightarrow{J} \sigma_i u_i.$$

Any input decomposes as $x = \sum_i a_i v_i$ (since V is orthogonal), so

$$Jx = \sum_i a_i \sigma_i u_i.$$

The map (1) decomposes into input directions v_i, (2) stretches each by σ_i, (3) re-expresses it in output direction u_i. A direction with $\sigma_i = 10$ grows amplitude $10\times$ and **variance** $100\times$; a direction with $\sigma_i = 0.1$ is suppressed. So the honest statement is:

Jacobians don't uniformly amplify noise; they stretch some direction

That is why noise injected as a JVP tangent is risky in a way a plain label is not.

9.6 SVD vs PCA

These are easy to conflate. PCA asks *which directions in a dataset carry the most variance*; the SVD of a Jacobian asks *which input directions this map stretches most*. They connect through

$$J^\top J = V\Sigma^2 V^\top,$$

so the right-singular vectors v_i are eigenvectors of $J^\top J$ with eigenvalues $\lambda_i = \sigma_i^2$. But call them **singular directions**, not PCA components — you are studying the transformation J, not a data distribution.

9.7 Label noise vs tangent noise: why iMF's reversal helps

Standard flow matching puts the noise only in the *label*:

$$L = \|v_\theta - v_c\|^2, \qquad v_c = v + \eta,$$

and MSE averages it out: $\mathbb{E}[v_c \mid z] = v$. Original MeanFlow instead feeds the *same* noisy sample into a learned JVP, turning it into $J_z u_\theta\, \eta$, which the Jacobian can distort (§9.4–9.5). The structural distinction:

ordinary regression: v_c is only a label vs original MF: v_c is also an i

Intermediate fix (useful but incomplete). Replace the noisy tangent with a model estimate of the marginal field, $v_\theta(z,t) = u_\theta(z,t,t)$ (the diagonal limit $r \to t$, §6.17), giving $u_{\text{tgt}} = v_c - \Delta[J_z u_\theta\, v_\theta + \partial_t u_\theta]$. This removes the noisy v_c from the tangent — a real improvement — but u_{tgt} still contains $\dot{u}_\theta$, so the target still moves (§9.1).

iMF's reversal. Use the *other* arrangement of the identity, $v = u + \Delta \dot{u}$, and treat the whole left side as the prediction:

$$V_\theta = u_\theta + \Delta\, \dot{u}_\theta, \qquad L_{\text{iMF}} = \mathbb{E}\|V_\theta - v_c\|^2.$$

Now the regression target is the *fixed, data-derived* label v_c; the population optimum is $V_\theta = \mathbb{E}[v_c \mid z, t] = v$. The predictor still moves (you are training it) but the thing it aims at is nailed down by the path construction — a cleaner statistical problem, while u_θ remains the underlying one-step-capable parameterization.

9.8 CFG inside original MeanFlow

How does original MeanFlow bake guidance into a one-step model? Take a labeled example (x, c), sample ϵ, t, r, form $z_t = (1-t)x + t\epsilon$, and take the raw sampled velocity $v_{\text{sample}} = \epsilon - x$. Estimate the *unconditional* instantaneous velocity from the network on the diagonal with the class dropped,

$$v_u^\theta = u_\theta(z_t, t, t, \varnothing),$$

then build the guided sample

$$v_g = v_u^\theta + \omega\,(v_{\text{sample}} - v_u^\theta).$$

Because $\mathbb{E}[v_{\text{sample}} \mid z_t, c] = v_{\text{cond}}$, its conditional mean is $v_u + \omega(v_{\text{cond}} - v_u)$ — exactly CFG. So v_g is a stochastic sample of the desired guided instantaneous field, and MeanFlow learns the average of that field.

9.9 Two independent axes: conditional/unconditional vs $r = t$ / $r < t$

A subtlety that is easy to garble: class dropout and the diagonal query are **different** axes. For a subset of examples the class is dropped; those use the raw v_{sample} as the target (unconditional, *no* CFG), while conditional examples use v_g. Separately, the diagonal evaluation $u_\theta(z_t, t, t, \varnothing)$ with $r = t$ is only an *auxiliary* query to estimate instantaneous unconditional velocity — it is **not** "the guided case." The main prediction is still $u_\theta(z_t, r, t, y)$ with $r \le t$. Keep the axes separate:

$$\underbrace{\text{conditional vs unconditional}}_{\text{is } c \text{ dropped?}} \qquad \text{and} \qquad \underbrace{r = t \text{ vs } r < t}_{\text{diagonal query vs interval}}.$$

9.10 Why original MF bakes in the CFG scale

In multistep diffusion you pick ω at each inference step and recompute $v_{\text{CFG}}^{(\omega)}$ on the fly. MeanFlow cannot: it learns a one-step *average* $u^{(\omega)}$, and changing ω changes the guided field $v^{(\omega)}$, which changes the trajectory $z^{(\omega)}$, which changes

the entire integral. Hence

$$u^{(2)} \neq u^{(5)} \quad \text{in general,}$$

so original MeanFlow effectively bakes the single training-time ω_0 into the learned conditional mean flow — a loss of test-time flexibility that iMF targets.

9.11 iMF makes ω an input: embedding and in-context conditioning

iMF learns the whole *family* $\omega \mapsto u^{(\omega)}$ by conditioning the network on the guidance strength,

$$u_\theta(z, r, t, c, \omega, \dots), \qquad \omega \sim p(\omega) \text{ during training,}$$

so the same image/class recurs with different requested ω, and inference picks $\omega \in \{2, 4, 8, \dots\}$ with no retraining. Since ω is a scalar but the transformer works in $\mathbb{R}^{d_{\text{model}}}$, it is embedded like a timestep: $\omega \to e_\omega$ via Fourier/sinusoidal features $\to$ MLP. iMF feeds these through **in-context conditioning** — conditions become tokens concatenated with the latent tokens rather than only driving AdaLN-style scale/shift:

$$[\, e_c,\ e_t,\ e_r,\ e_\omega,\ e_{t_{\min}},\ e_{t_{\max}},\ z_1, \dots, z_N \,],$$

so image tokens attend directly to condition tokens. The authors report this handles many heterogeneous conditions while reducing model size.

9.12 The guidance interval $[t_{\min}, t_{\max}]$ and 1-NFE

The deepest subtlety: why pass a *time interval* to a *one-step* model? Because u is defined as the average of an underlying continuous field, and you can make CFG active on only part of that virtual trajectory:

$$v_{\text{guided}}(z_t, t) = \begin{cases} v_{\text{CFG}}(z_t, t), & t \in [t_{\min}, t_{\max}], \\ v_{\text{base}}(z_t, t), & \text{otherwise,} \end{cases} \qquad u^{(\omega, t_{\min}, t_{\max})} = \frac{1}{t - r} J$$

The one-step network learns the *total displacement* that this virtual schedule would produce — it never traverses those times at inference. This is the entire MeanFlow philosophy: **learn the result of integrating a trajectory instead of integrating it**. The analogy: "drive Boston→New York, using the highway only between mile 50 and 150" can be answered by a map that predicts the final displacement directly, without simulating every mile. Released iMF checkpoints are evaluated at **1 NFE** with, e.g., $\omega = 8$, $t_{\min} = 0.4$, $t_{\max} = 0.65$.

9.13 Guidance interval vs number of inference steps

These are orthogonal variables and must not be conflated:

$[t_{\min}, t_{\max}]$ = where guidance is conceptually active on the underlying

NFE = how many network evaluations you actually run to generate.

So $t_{\min} = 0.4,\ t_{\max} = 0.65$ with $\mathrm{NFE} = 1$ is perfectly consistent — and is exactly how iMF is evaluated.

9.14 The full FM → MF → modified MF → iMF ladder

$$\textbf{FM:}\quad v_\theta(z,t) \to v_c, \qquad v_\theta^* = v, \qquad \text{integrate } \tfrac{dz}{dt} = v_\theta \text{ (multistep)}.$$

$$\textbf{MF:}\quad u_{\text{tgt}} = v_c - \Delta\,\mathrm{JVP}(u_\theta; v_c), \qquad u_\theta \to \mathrm{sg}(u_{\text{tgt}}).$$

Enables one-step $z_0 = z_1 - u(z_1, 0, 1)$; but noisy v_c enters the JVP, the target moves, and ω is fixed at train time.

$$\textbf{Modified MF:}\quad u_{\text{tgt}} = v_c - \Delta\,\mathrm{JVP}(u_\theta; v_\theta), \quad v_\theta = u_\theta(z,t,t).$$

Removes the noisy tangent, but the target still moves.

$$\textbf{iMF:}\quad V_\theta = u_\theta + \Delta\,\mathrm{JVP}(u_\theta; v_\theta) \to v_c, \qquad V_\theta^* = v.$$

Core target is FM's fixed data label, yet u_θ stays the one-step parameterization — and $(\omega, t_{\min}, t_{\max})$ are inputs. The one-sentence compression:

FM: learn the local arrow v; **MF: learn the average arrow** u;

> **Key takeaway.** Stop-gradient only freezes MeanFlow's target *within* an update — across updates the target $v_c - \Delta\dot{u}_{\theta_k}$ still drifts, so original MF is bootstrapped regression toward a moving target. The second, independent flaw is that the *noisy* conditional label $v_c = v + \eta$ enters the JVP as a tangent, where the Jacobian's singular directions (SVD $J = U\Sigma V^\top$, variance scaled by σ_i^2) can amplify it — harmless as a label, harmful as a tangent. iMF fixes both by reversing the identity to $V_\theta = u_\theta + \Delta\dot{u}_\theta \to v_c$ (fixed data label, optimum v) while keeping u_θ one-step-capable, and by making guidance $(\omega, t_{\min}, t_{\max})$ *inputs* via in-context condition tokens. A one-step model can still honor a guidance *interval* because it learns the integral of a virtual guided flow — $t_{\min}{=}0.4, t_{\max}{=}0.65$ at $\mathrm{NFE}{=}1$ is not a contradiction.